PHP Cookies

What is a Cookie?

A cookie is often used to **identify a user**. A cookie is a **small file** that the server embeds on the user's computer. Each time the same computer requests a page with a browser, it will send the cookie too. With PHP, you can both create and retrieve cookie values.

Create Cookies With PHP

A cookie is created with the **setcookie()** function.

Syntax

setcookie(*name, value, expire, path, domain, secure, httponly*);

Only the ***name*** **parameter is required**. All other parameters are optional.

1.PHP Create/Retrieve a Cookie

The following example creates a cookie named "user" with the value "John Doe". The cookie will expire after 30 days (86400 * 30). The "/" means that the cookie is available in entire website (otherwise, select the directory you prefer).

We then retrieve the value of the cookie "user" (using the global variable $_COOKIE). We also use the isset() function to find out if the cookie is set:

Example :

```
<?php
$cookie_name = "user";
$cookie_value = "John Doe";
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/"); // 86400 = 1
day
?>
<html>
<body>

<?php
if(!isset($_COOKIE[$cookie_name])) {
  echo "Cookie named '" . $cookie_name . "' is not set!";
} else {
  echo "Cookie '" . $cookie_name . "' is set!<br>";
  echo "Value is: " . $_COOKIE[$cookie_name];
}
?>

</body>
```

</html>

Note: The setcookie() function must appear BEFORE the <html> tag.

Note: The value of the cookie is automatically URLencoded when sending the cookie, and automatically decoded when received (to prevent URLencoding, use setrawcookie() instead).

database_php > cookie_php > createcookie.php > html > body

```php
<?php
$cookie_name = "user";
$cookie_value = "subash";
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/"); // 86400 = 1 day
?>
<html>
<body>

<?php
if(!isset($_COOKIE[$cookie_name])) {
  echo "Cookie named '" . $cookie_name . "' is not set!";
} else {
  echo "Cookie '" . $cookie_name . "' is set!<br>";
  echo "Value is: " . $_COOKIE[$cookie_name];
}
?>
```

localhost/fifth_php/database_php/cookie_php/createcook...

Cookie named 'user' is not set!

localhost/fifth_php/database_php/cookie_php/createcookie.php

Cookie 'user' is set!
Value is: subash

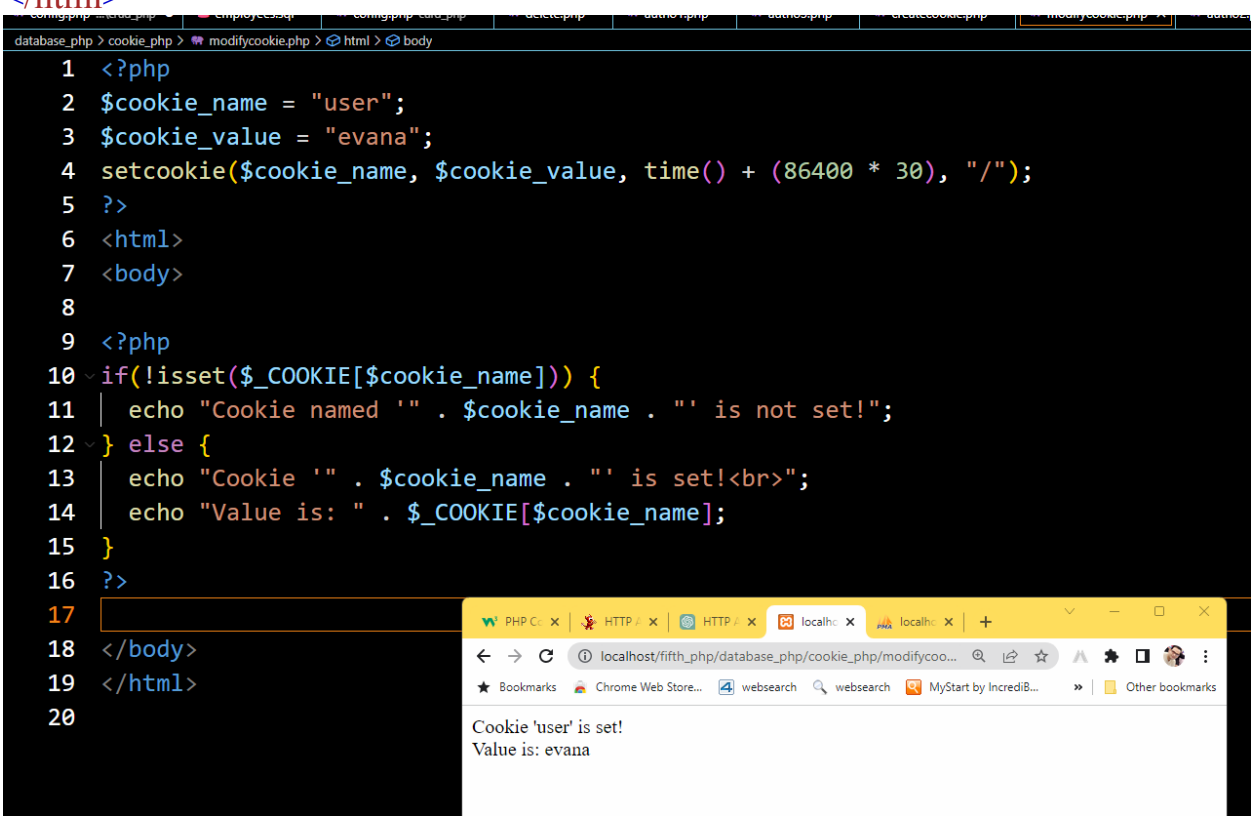
Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	SamePa...	Partition Key	Prio...
user	subash	localhost	/	2023-03-15T12:03:25.181Z	10						Medi...

2.Modify a Cookie Value

To modify a cookie, just set (again) the cookie using the setcookie() function:

Example

```
<?php
$cookie_name = "user";
$cookie_value = "Alex Porter";
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/");
?>
<html>
<body>

<?php
if(!isset($_COOKIE[$cookie_name])) {
  echo "Cookie named '" . $cookie_name . "' is not set!";
} else {
  echo "Cookie '" . $cookie_name . "' is set!<br>";
  echo "Value is: " . $_COOKIE[$cookie_name];
}
?>
</body>
</html>
```

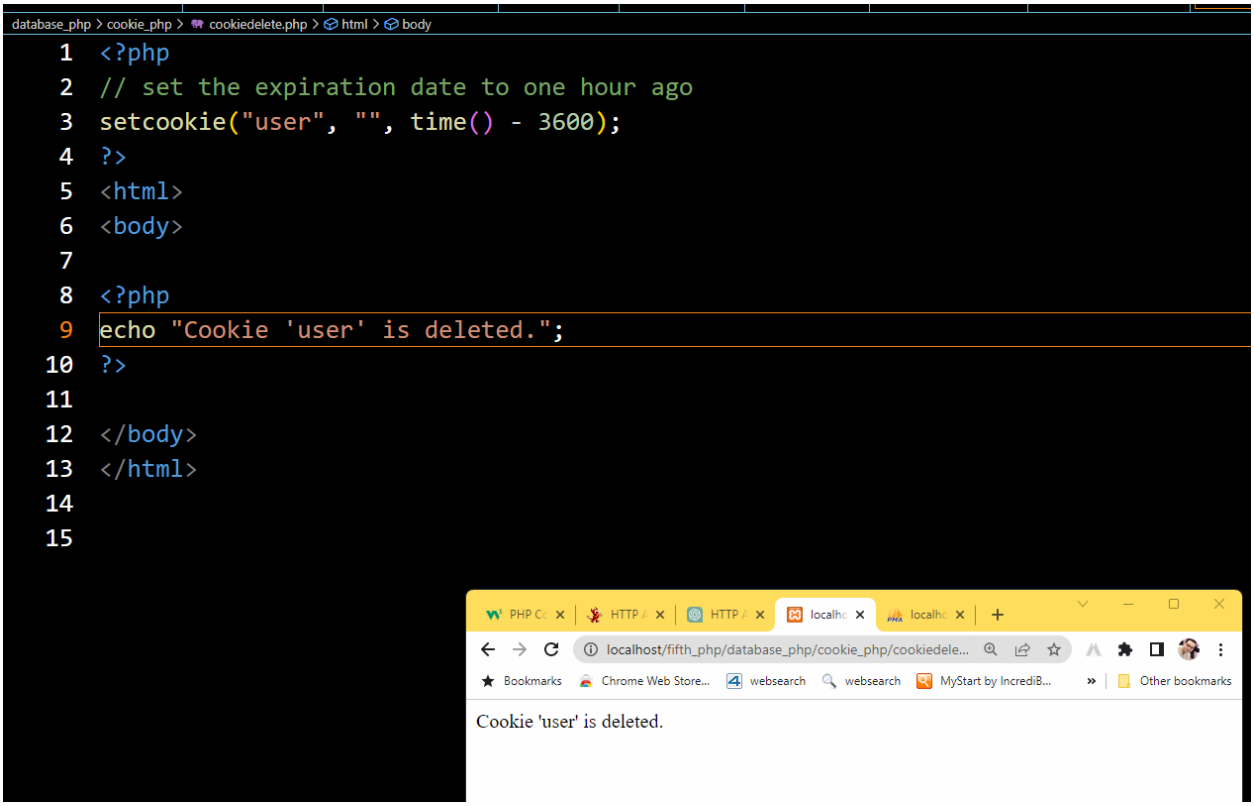


3.Delete a Cookie

To delete a cookie, use the setcookie() function with an expiration date in the past:

Example

```
<?php
// set the expiration date to one hour ago
setcookie("user", "", time() - 3600);
?>
<html>
<body>

<?php
echo "Cookie 'user' is deleted.";
?>

</body>
</html>
```

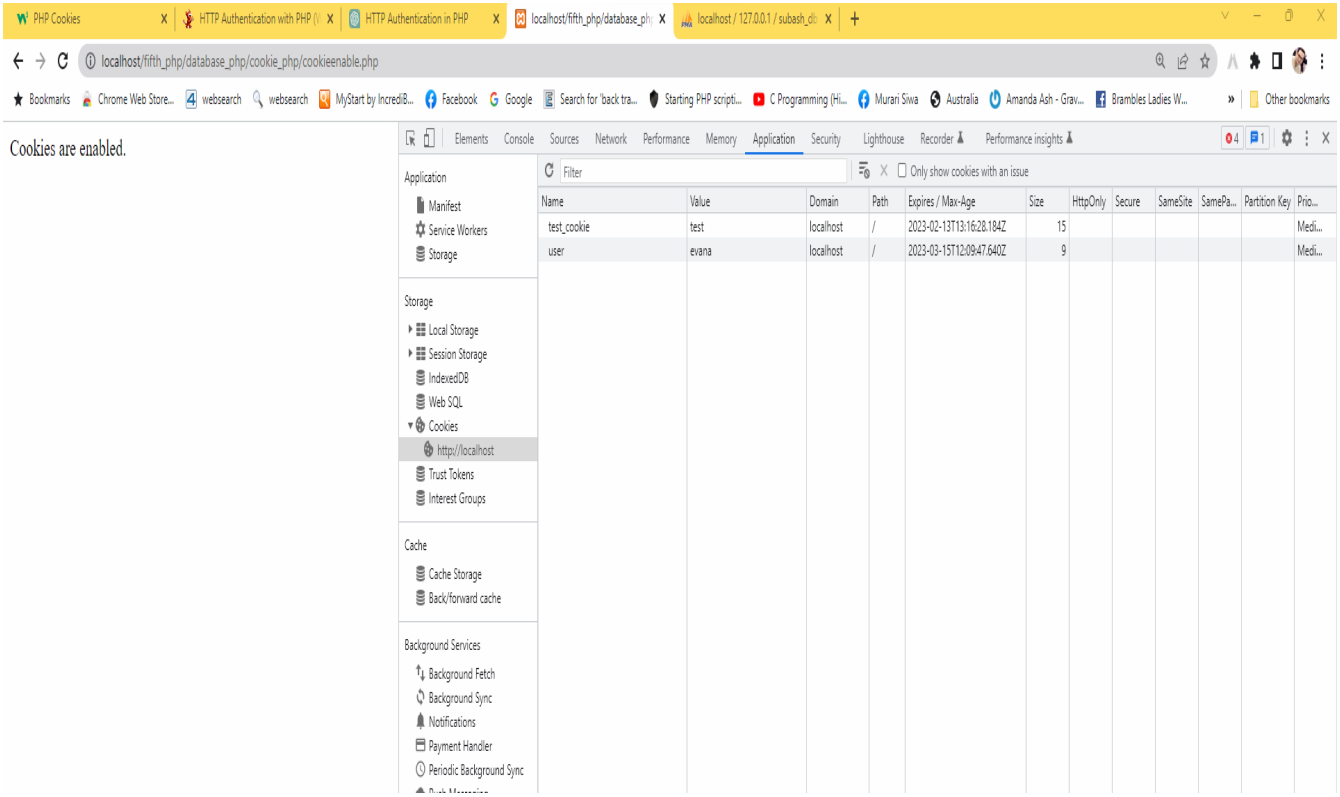


4.Check if Cookies are Enabled

The following example creates a small script that checks whether cookies are enabled. First, try to create a test cookie with the setcookie() function, then count the $_COOKIE array variable:

Example

```
<?php
setcookie("test_cookie", "test", time() + 3600, '/');
?>
<html>
<body>

<?php
if(count($_COOKIE) > 0) {
  echo "Cookies are enabled.";
} else {
  echo "Cookies are disabled.";
}
?>

</body>
</html>
```

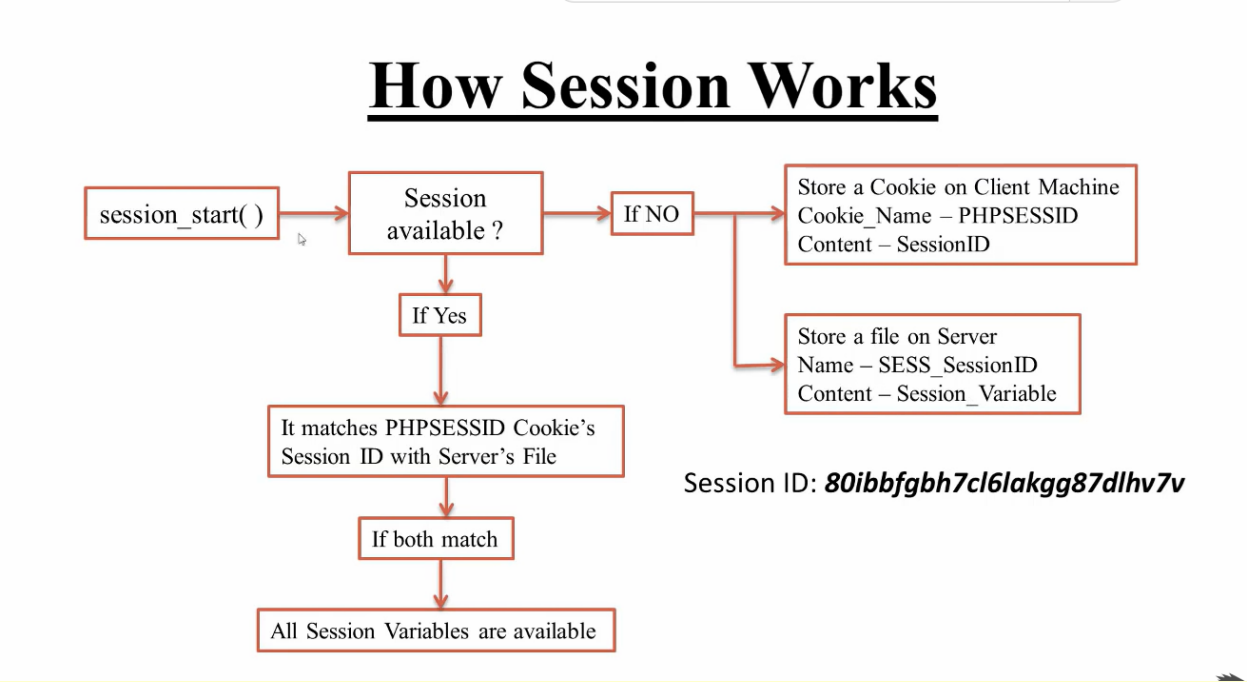


PHP Sessions

A session is a way to store information (in variables) to be used across multiple pages.
Unlike a cookie, the information is not stored on the users computer.

What is a PHP Session?

When you work with an application, you open it, do some changes, and then you close it. This is much like a Session. The computer knows who you are. It knows when you start the application and when you end. But on the internet there is one problem: the web server does not know who you are or what you do, because the HTTP address doesn't maintain state.
Session variables solve this problem by storing user information to be used across multiple pages (e.g. username, favorite color, etc). By default, session variables last until the user closes the browser.So; Session variables hold information about one single user, and are available to all pages in one application.
Tip: If you need a permanent storage, you may want to store the data in a database.



1.Start a PHP Session

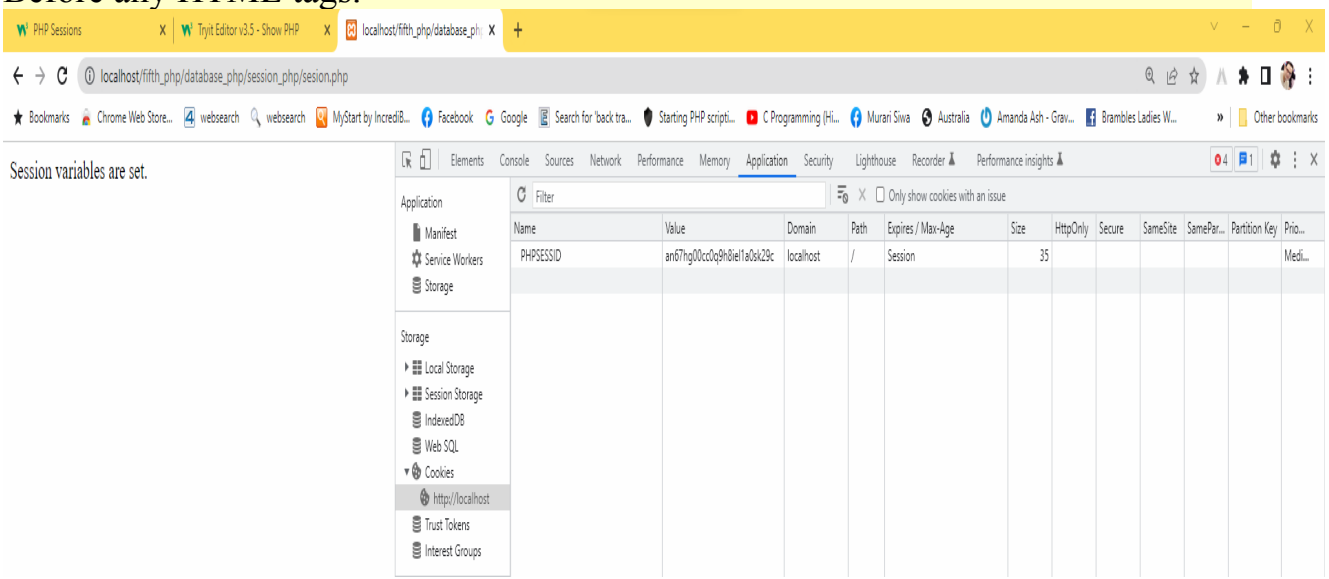
A session is started with the session_start() function.
Session variables are set with the PHP global variable: $_SESSION.
Now, let's create a new page called "demo_session1.php". In this page, we start a new PHP session and set some session variables:

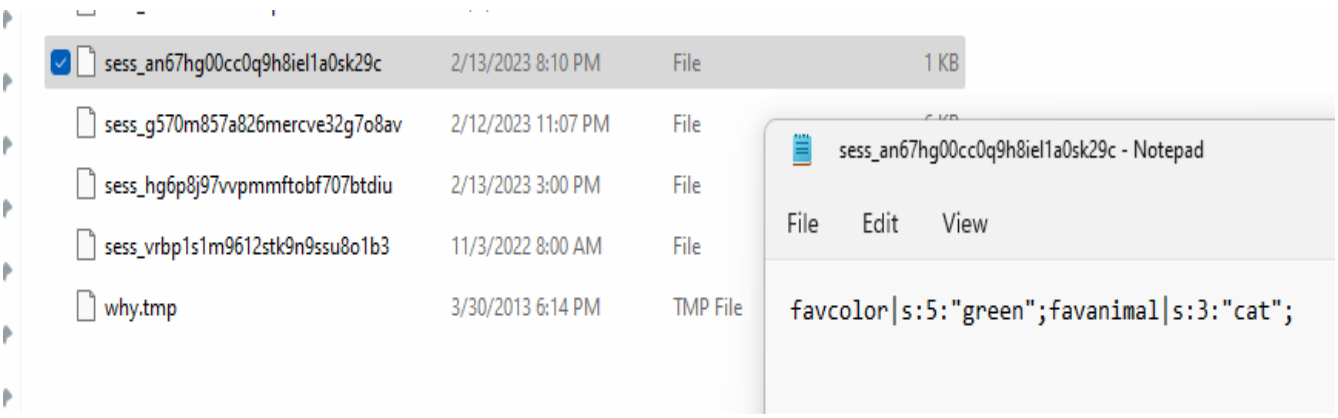
Example

```
<?php
```

```php
// Start the session
session_start();
// Set session variables
$_SESSION["favcolor"] = "green";
$_SESSION["favanimal"] = "cat";
echo "Session variables are set.";
?>
```

Note: The session_start() function must be the very first thing in your document. Before any HTML tags.



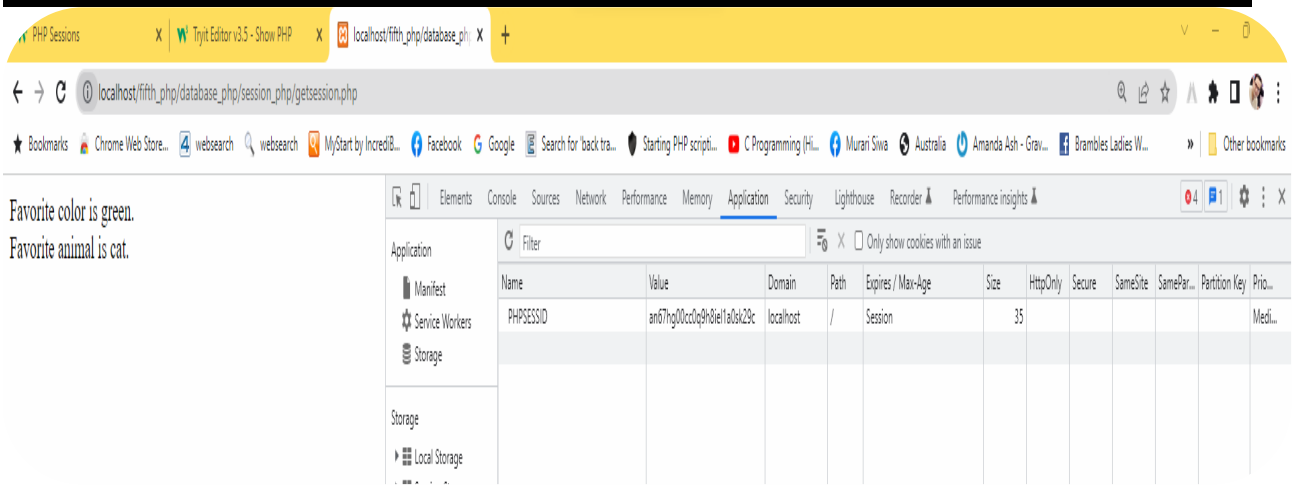


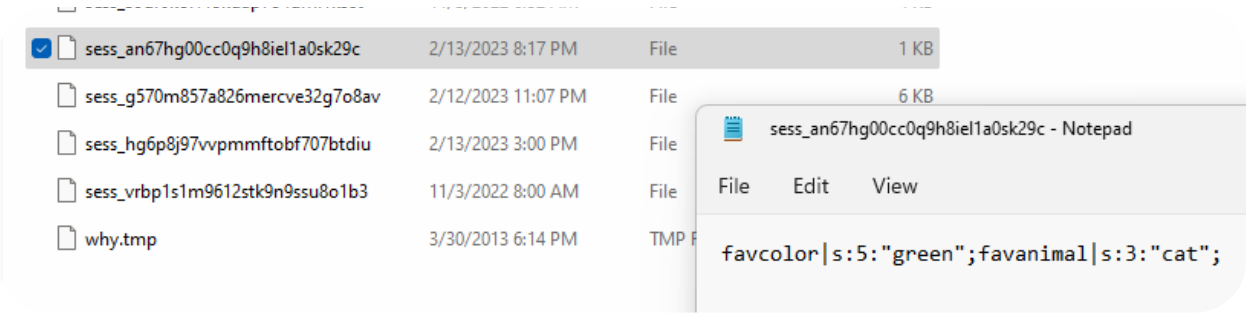
2.Get PHP Session Variable Values

Next, we create another page called "demo_session2.php". From this page, we will access the session information we set on the first page ("demo_session1.php").
Notice that session variables are not passed individually to each new page, instead they are retrieved from the session we open at the beginning of each page (session_start()).
Also notice that all session variable values are stored in the global $_SESSION variable:

Example

```
<?php
// Start the session
session_start();
// Echo session variables that were set on previous page
echo "Favorite color is " . $_SESSION["favcolor"] . ".<br>";
echo "Favorite animal is " . $_SESSION["favanimal"] . ".";
?>
```





Another way to show all the session variable values for a user session is to run the following code:

Example

```
<?php
session_start();
print_r($_SESSION);
?>
```

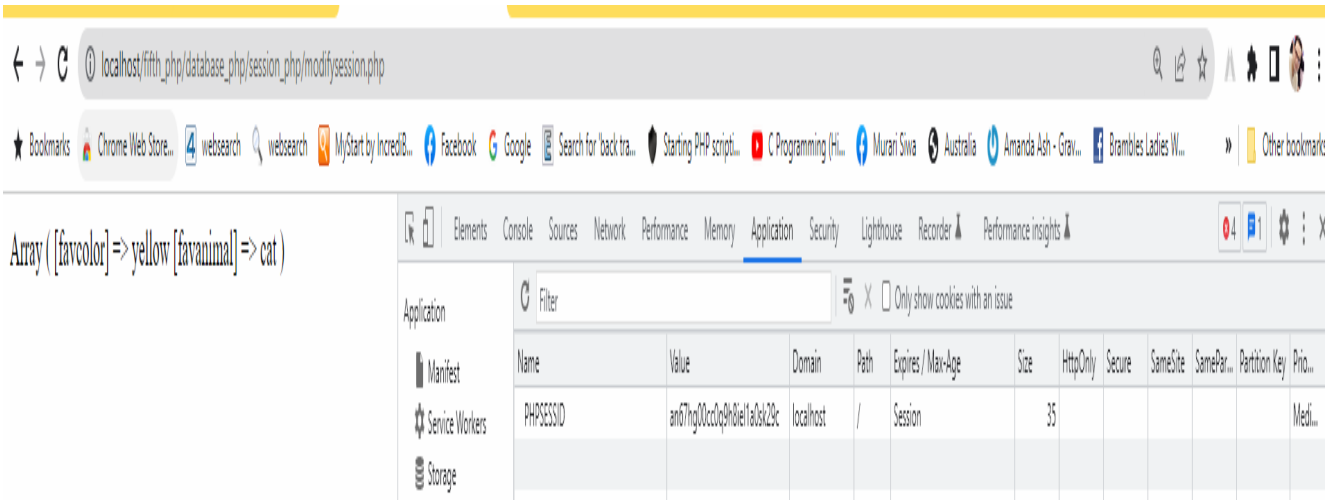
How does it work? How does it know it's me?

Most sessions set a user-key on the user's computer that looks something like this: sess_vrbp1s1m9612stk9n9ssu8o1b3. Then, when a session is opened on another page, it scans the computer for a user-key. If there is a match, it accesses that session, if not, it starts a new session.

3.Modify a PHP Session Variable

```
<?php

session_start();



// to change a session variable, just
overwrite it

$_SESSION "favcolor"  = "yellow"
```

```
print_r($_SESSION);
?>
```



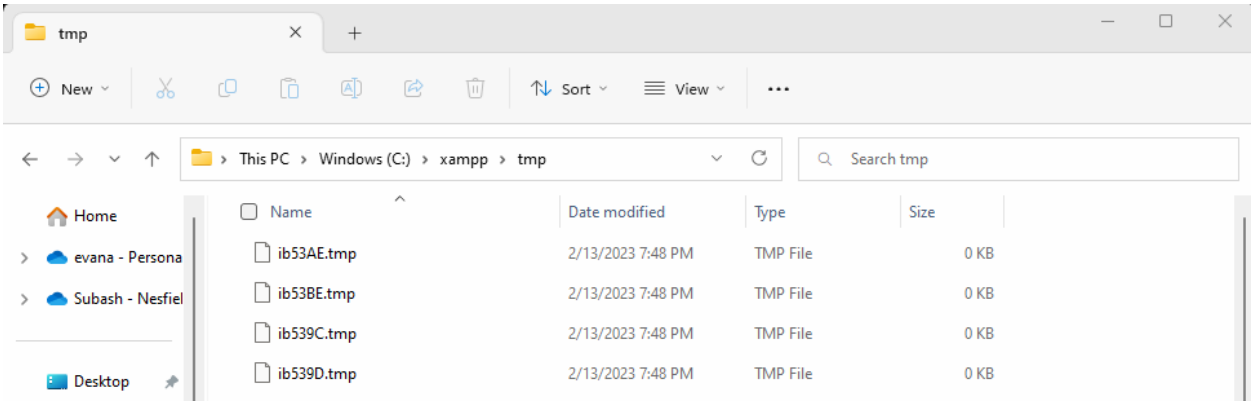
4.Destroy a PHP Session

To remove all global session variables and destroy the session, use session_unset() and session_destroy():

Example

```
<?php
session_start();
?>
<?php
// remove all session variables
session_unset();

// destroy the session
session_destroy();
```

```
?>
```



HTTP Authentication in php

HTTP authentication is a way to secure your web pages or resources by prompting the user to enter their username and password. In PHP, you can implement HTTP authentication by using the header function.

Here's an example:

```
<?php
if (!isset($_SERVER['PHP_AUTH_USER'])) {
    header('WWW-Authenticate: Basic realm="My Realm"');
    header('HTTP/1.0 401 Unauthorized');
    echo 'Text to send if user hits Cancel button';
    exit;
} else {
    echo "<p>Hello {$_SERVER['PHP_AUTH_USER']}.</p>";
    echo "<p>You entered {$_SERVER['PHP_AUTH_PW']} as your
password.</p>";
}
?>
```

In this example, the header function is used to send the WWW-Authenticate header, which tells the browser to prompt the user for their credentials. The realm attribute specifies the name of the authentication realm. The second header call is

used to return a 401 Unauthorized status code, indicating that authentication is required.

When the user visits the page, they will be prompted to enter their username and password. If they enter the correct credentials, the rest of the script will be executed and the information will be displayed. If they cancel the prompt, the text "Text to send if user hits Cancel button" will be displayed.

Keep in mind that HTTP authentication is not very secure, as the credentials are transmitted in plain text. If you need to secure sensitive information, it's recommended to use HTTPS and a more secure authentication method, such as OAuth.

examples in HTTP Authentication

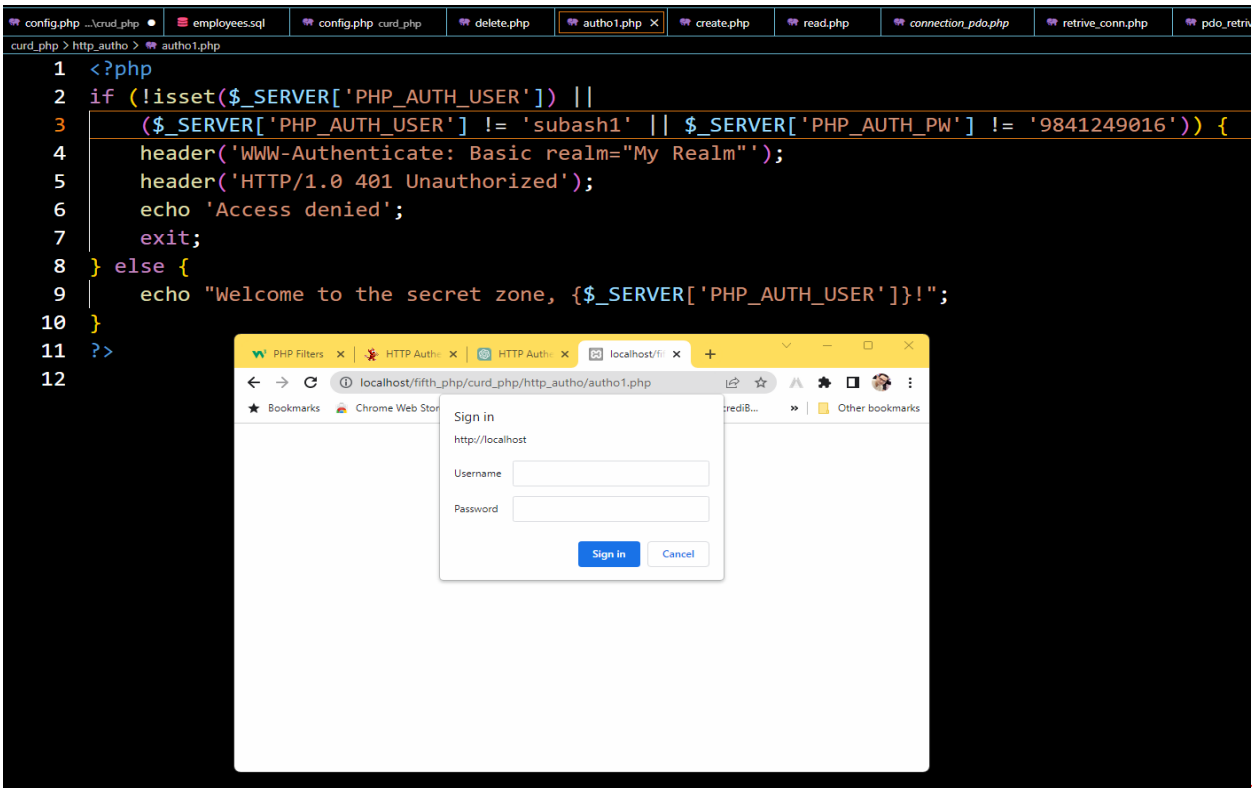
Here are a few examples that demonstrate different aspects of HTTP authentication in PHP:

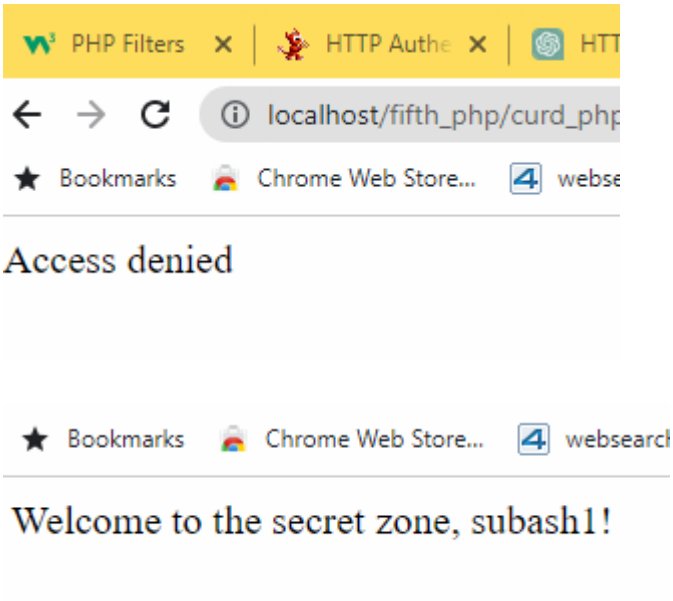
1.Basic HTTP Authentication:

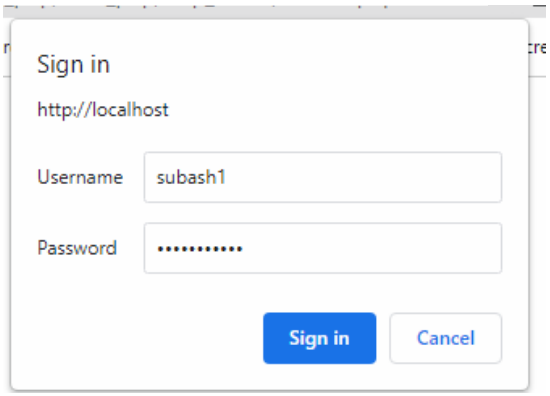
```
<?php
if (!isset($_SERVER['PHP_AUTH_USER']) ||
   ($_SERVER['PHP_AUTH_USER'] != 'username' ||
$_SERVER['PHP_AUTH_PW'] != 'password')) {
   header('WWW-Authenticate: Basic realm="My Realm"');
   header('HTTP/1.0 401 Unauthorized');
   echo 'Access denied';
   exit;
} else {
   echo "Welcome to the secret zone, {$_SERVER['PHP_AUTH_USER']}!";
}
?>
```

This is a simple example of basic HTTP authentication. The user will be prompted to enter their credentials, and if they enter the correct username and password, they will be granted access to the secret zone.

Example:





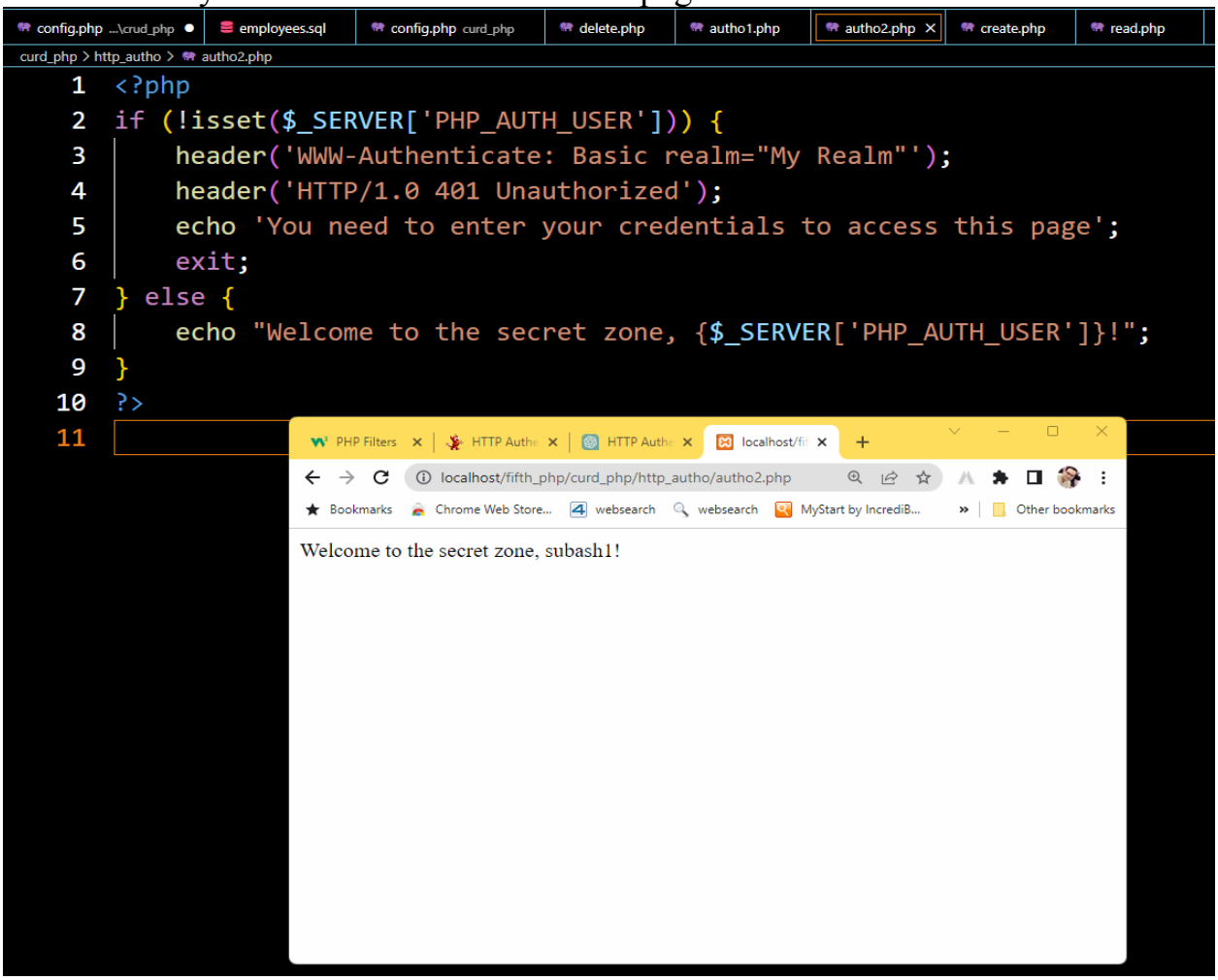


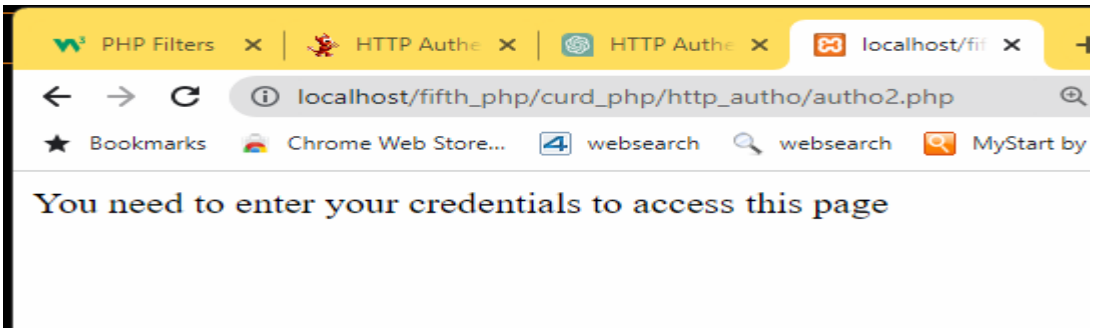
2.Customizing the authentication prompt:

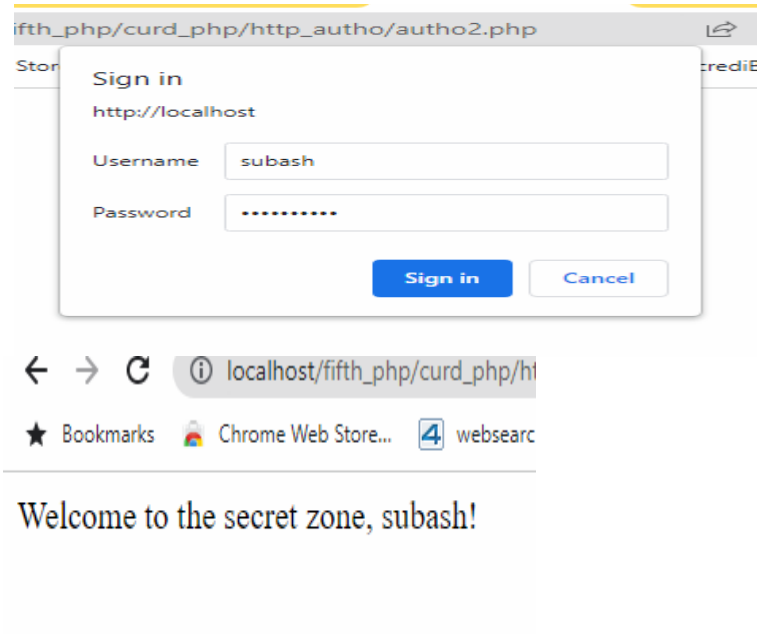
```
<?php
if (!isset($_SERVER['PHP_AUTH_USER'])) {
    header('WWW-Authenticate: Basic realm="My Realm"');
    header('HTTP/1.0 401 Unauthorized');
    echo 'You need to enter your credentials to access this page';
```

```
    exit;
} else {
    echo "Welcome to the secret zone, {$_SERVER['PHP_AUTH_USER']}!";
}
?>
```

In this example, we're using a custom message to provide additional information to the user. If they cancel the authentication prompt, they will see the message "You need to enter your credentials to access this page".

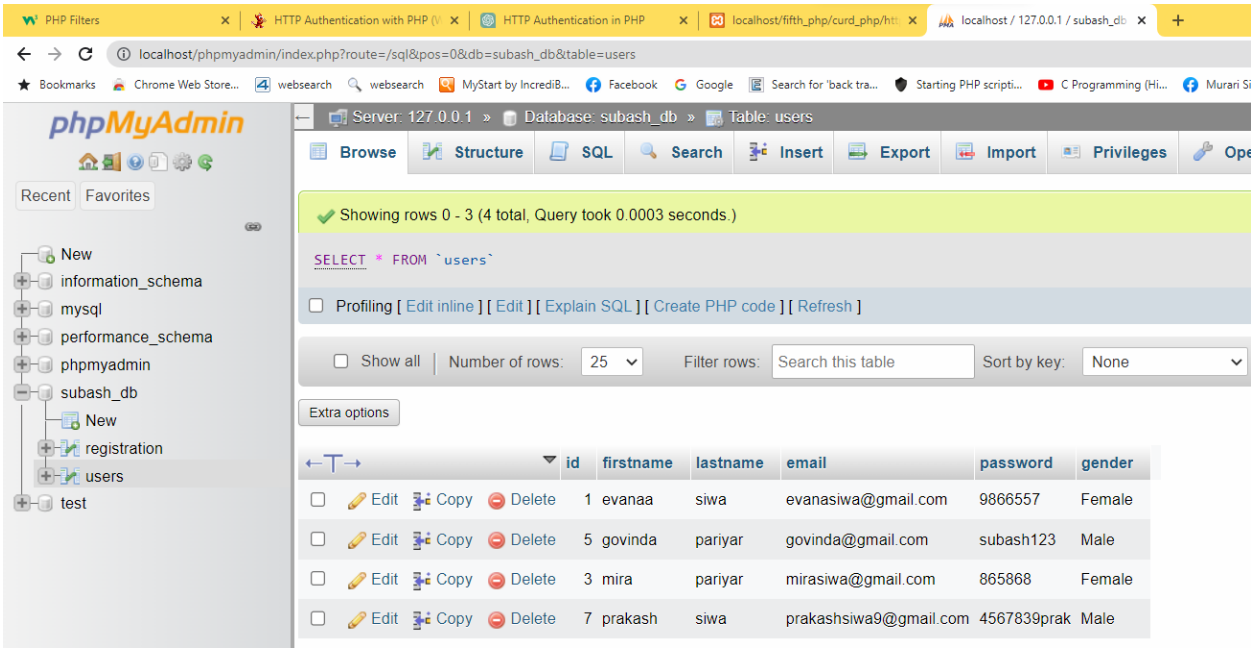


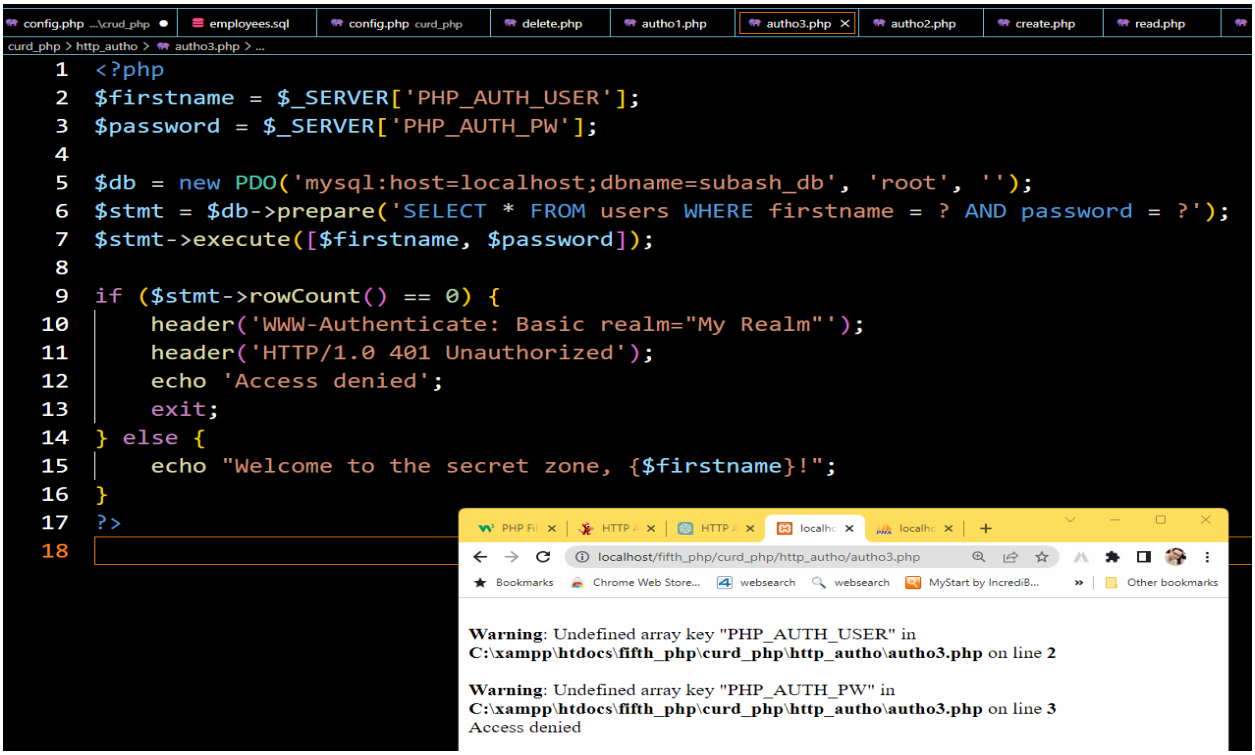


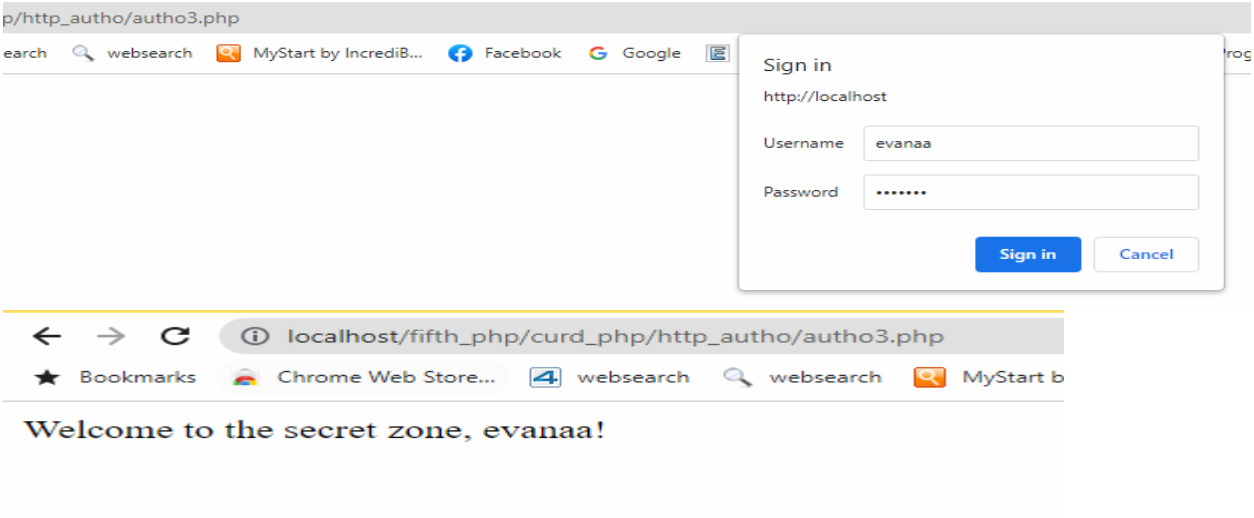


3.Checking the credentials against a database:

```
<?php
$username = $_SERVER['PHP_AUTH_USER'];
$password = $_SERVER['PHP_AUTH_PW'];

$db = new PDO('mysql:host=localhost;dbname=mydb', 'dbuser', 'dbpassword');
$stmt = $db->prepare('SELECT * FROM users WHERE username = ? AND
password = ?');
$stmt->execute([$username, $password]);

if ($stmt->rowCount() == 0) {
    header('WWW-Authenticate: Basic realm="My Realm"');
    header('HTTP/1.0 401 Unauthorized');
    echo 'Access denied';
    exit;
} else {
    echo "Welcome to the secret zone, {$username}!";
}
?>
```


localhost/phpmyadmin/index.php?route=/sql&pos=0&db=subash_db&table=users
phpMyAdmin
Recent Favorites
New
information_schema
mysql
performance_schema
phpmyadmin
subash_db
New
registration
users
test
Server: 127.0.0.1 » Database: subash_db » Table: users
Browse Structure SQL Search Insert Export Import Privileges
Showing rows 0 - 3 (4 total, Query took 0.0003 seconds.)
SELECT * FROM `users`
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None
Extra options
id firstname lastname email password gender
Edit Copy Delete 1 evanaa siwa evanasiwa@gmail.com 9866557 Female
Edit Copy Delete 5 govinda pariyar govinda@gmail.com subash123 Male
Edit Copy Delete 3 mira pariyar mirasiwa@gmail.com 865868 Female
Edit Copy Delete 7 prakash siwa prakashsiwa9@gmail.com 4567839prak Male


config.php employees.sql delete.php autho1.php autho3.php autho2.php create.php read.php
curd_php > http_autho > autho3.php > ...
<?php
$firstname = $_SERVER['PHP_AUTH_USER'];
$password = $_SERVER['PHP_AUTH_PW'];
$db = new PDO('mysql:host=localhost;dbname=subash_db', 'root', '');
$stmt = $db->prepare('SELECT * FROM users WHERE firstname = ? AND password = ?');
$stmt->execute([$firstname, $password]);
if ($stmt->rowCount() == 0) {
header('WWW-Authenticate: Basic realm="My Realm"');
header('HTTP/1.0 401 Unauthorized');
echo 'Access denied';
exit;
} else {
echo "Welcome to the secret zone, {$firstname}!";
}
?>
localhost/fifth_php/curd_php/http_autho/autho3.php
Warning: Undefined array key "PHP_AUTH_USER" in C:\xampp\htdocs\fifth_php\curd_php\http_autho\autho3.php on line 2
Warning: Undefined array key "PHP_AUTH_PW" in C:\xampp\htdocs\fifth_php\curd_php\http_autho\autho3.php on line 3
Access denied



In this example, we're using a database to store the users and their credentials. After the user enters their credentials, we check the values against the database and grant access only if the username and password match.

These are just a few examples of how you can use HTTP authentication in PHP. You can also use other authentication methods, such as OAuth, if you need a more secure solution.